Advanced Data Structures

Pritha Banerjee

University of Calcutta

banerjee.pritha74@gmail.com

Overview

1. Standard tries
2. Compressed tries / Patricia tries
3. Books etc.

Table of Contents

1. Standard tries
2. Compressed tries / Patricia tries
3. Books etc.

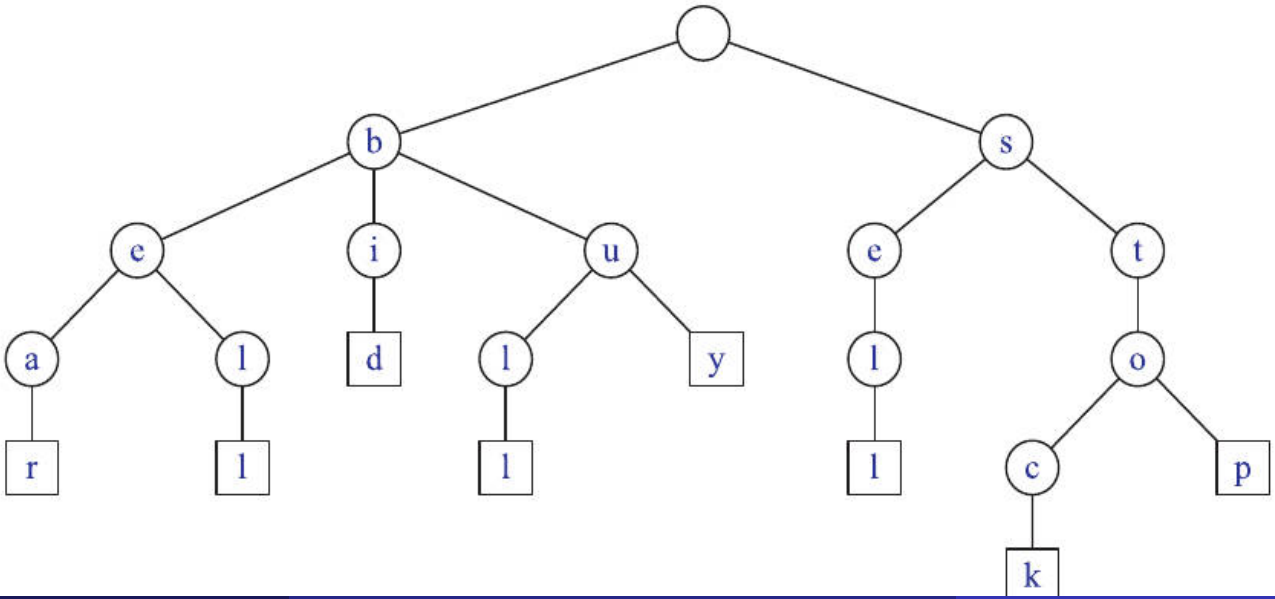
Tries

- **Pattern matching** algorithms such as KMP, finite automata based matcher speed up the search in a text by **preprocessing the pattern**
- **A complementary approach:** string searching algorithms that **preprocess the text**.
- **Application:** where a series of **queries** is performed **on a fixed text**, so that the initial cost of preprocessing the text is compensated by a speedup in each subsequent query
- **Example:** a Web site that offers pattern matching in Shakespeare's Hamlet or a search engine that offers Web pages on the Hamlet topic or auto completion feature in any text input.
- A **trie** (pronounced "try") is a tree-based data structure for storing strings in order to support fast pattern matching, mainly used in information re**trie**val.
- Given a collection S of strings, all defined on same alphabet, search for a certain DNA sequence.
- The primary query operations that **tries** support are **pattern matching** and **prefix matching** (search all strings in S that contain X as a prefix).

Standard tries : Properties

Let S be a set of s strings from alphabet Σ such that no string in S is a prefix of another string. A **standard trie for** S is an **ordered tree** T with properties:

- Each node of T, except the root, is labeled with a character of Σ (can have atmost d children where $d = |\Sigma|$)
- The ordering of children of an **internal node** of T is determined by a canonical ordering of the alphabet Σ.
- T has s external (leaf) nodes, each associated with a string of S, such that the concatenation of the labels of the nodes on the path from the root to a leaf node v of T yields the string of S associated with v. (ensures that each string of S is uniquely associated with a leaf of T).



Standard tries: structural properties

A **standard trie** storing a collection S of s strings of total length n from an alphabet of size d has following properties:

- Every internal node of T has at most d children
- T has s external nodes
- The height of T is equal to the length of the **longest string** in S
- The number of nodes of T is $O(n)$

Application- Word matching: Determine whether a given pattern matches one of the words of the text exactly.

- A trie T for a set S of strings can be used to implement a **dictionary**, whose keys are the strings of S.
- Search in T for a string X by tracing down from the root the path indicated by the characters in X.
- If this path can be traced and terminates at an external node, then X is in the dictionary.
- Time complexity of searching a string is $O(dm)$

Word Matching: Example

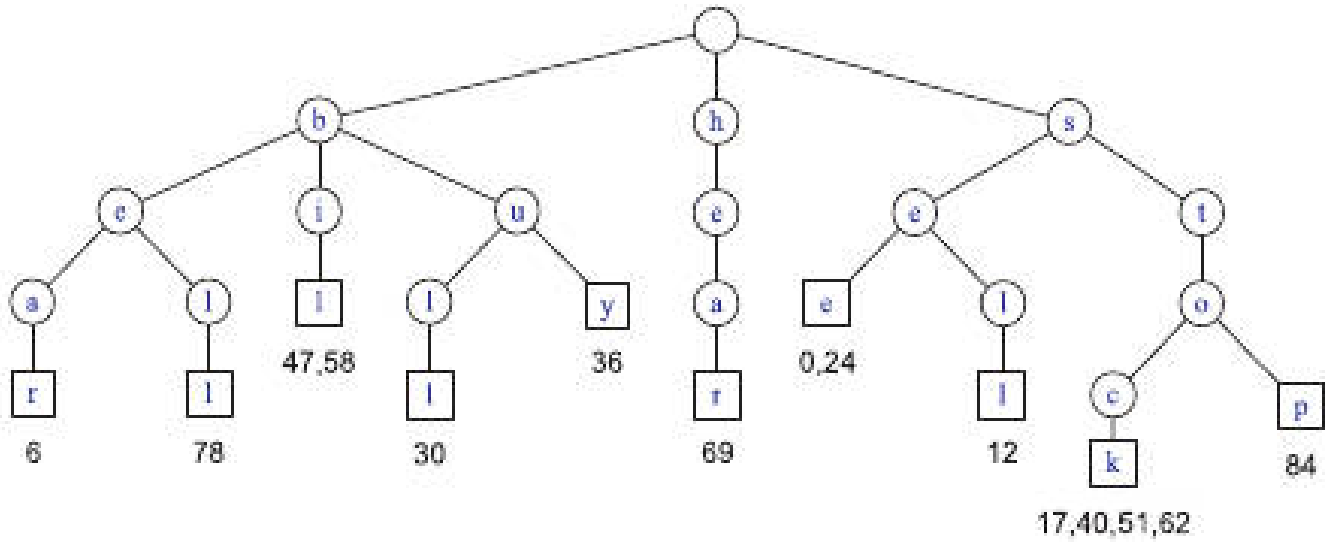
s	e	e		a		b	e	a	r	?		s	e	l	l		s	t	o	c	k	!	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23

s	e	e		a		b	u	l	l	?		b	u	y		s	t	o	c	k	!	
24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46

b	i	d		s	t	o	c	k	!		b	i	d		s	t	o	c	k	!	
47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68

h	e	a	r		t	h	e		b	e	l	l	?		s	t	o	p	!
69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88

(a)



a) Text to be searched; numbers below leaf in standard trie indicates the position of word in text

Building Standard trie

Use an **incremental algorithm** that inserts the strings from S one at a time.
Assumption: No string of S is a **prefix** of another string.

- To insert string X into current trie T, trace the path associated with X in T.
- Since X is not already in T and no string in S is a prefix of another string, tracing stops at an internal node v of T before reaching the end of X .
- Create a new chain of node descendents of v to store the remaining characters of X.
- Time to insert X is $O(dm)$, where m is the length of X and d is the size of the alphabet.
- Time complexity of constructing the entire trie for set S takes $O(dn)$ time, where n is the total length of the strings of S.

Limitations: there are many nodes with one child which wastes space. Thus, **Compressed tries** or **Patricia tries**

Table of Contents

1 Standard tries

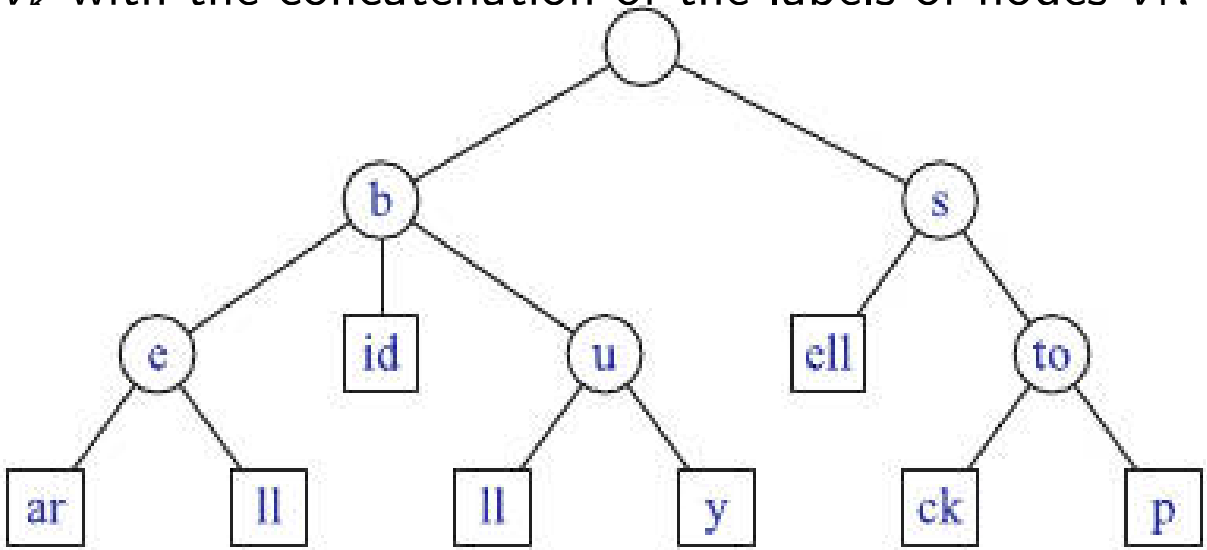
2 Compressed tries / Patricia tries

3 Books etc.

Compressed/ Patricia tries

A **compressed trie** ensures that each internal node in the trie has **at least two children** by compressing chains of single-child nodes into individual edges.

- An internal node v of standard trie T is **redundant** if v has **one child** and is not the root. A chain of $k \geq 2$ edges $(v_0, v_1)(v_1, v_2) \cdots (v_{k-1}, v_k)$ is redundant if:
  - v_i is redundant for $i = 1, \cdots, k-1$
  - v_0 and v_k are not redundant
- T can be transformed into a **compressed trie** by replacing each **redundant chain** $(v_0, v_1)(v_1, v_2) \cdots (v_{k-1}, v_k)$ of $k \geq 2$ edges into a single edge (v_0, v_k), relabeling v_k with the concatenation of the labels of nodes $v_1, \cdots, v_k$



Compressed trie for the strings {bear, bell, bid, bull, buy, sell, stock, stop}

Compressed tries: properties

A compressed trie storing a collection S of s strings from an alphabet of size d has the following properties:

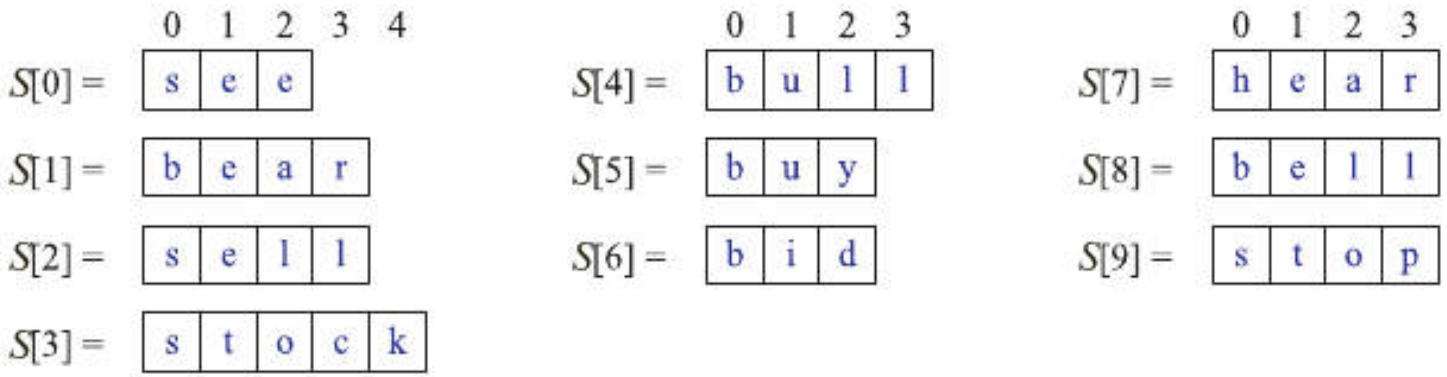
- Every internal node of T has at least two children and most d children
- T has s external nodes
- The number of nodes of T is $O(s)$

Is it advantageous? a compressed trie is truly advantageous only when it is used as an **auxiliary index structure** over a collection of strings **already stored in a primary structure**, and is not required to actually store all the characters of the strings in the collection.

Space complexity: reduce the total space for the trie itself from $O(n)$ for the standard trie to $O(s)$ for the compressed trie, where n is the total length of the strings in S and s is the number of strings in S, although strings in S must be stored.

Compressed tries: is it advantageous?

The collection S of strings is an array of strings $S[0], S[1], \cdots, S[s-1]$. Instead of storing the label X of a node explicitly, represent it implicitly by a **triplet** of integers (i, j, k), such that $X = S[i][j..k]$; that is, X is the substring of $S[i]$ consisting of the characters from the jth to the kth included.



(a)

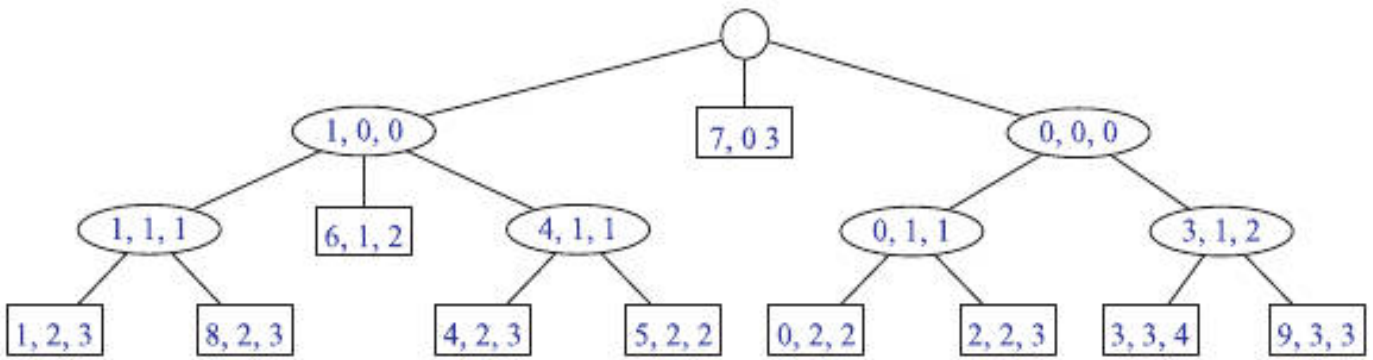


Table of Contents

1. Standard tries
2. Compressed tries / Patricia tries
3. Books etc.

References

 Michael T Goodrich, Roberto Tamassia, David M. Mount, Data Structures and Algorithms in C++, 2nd Edition, John Wiley and Sons, Inc, 2011

The End